

Adaptive Adversarial Training (AAT)

Setup, execution and monitoring guide for the Kaggle rebuild

Matoke Brian

repository github.com/mattobryan/AAT, branch `matt/funny-planck-3j7md5`

September 25, 2026

Contents

1	Purpose and research question	1
2	System overview	1
3	Repository layout	2
4	Step 1: the RAMP baseline	2
4.1	Targets	2
4.2	What is run	2
4.3	Acceptance	3
5	One-time setup	3
5.1	Hugging Face	3
5.2	Kaggle	3
5.3	Monitoring (Claude routine)	3
6	Running	3
6.1	A session	3
6.2	Resuming	4
6.3	Evaluating and comparing	4
7	Monitoring and reports	4
8	Step 2: AAT experiments	4
9	Evaluation protocol	5
10	Budget and schedule	5
11	Troubleshooting	5
12	Checklist	5
A	Command reference	6

1 Purpose and research question

This repository rebuilds the MSc thesis *Adaptive Adversarial Training: A Curriculum-Based Approach to Robustness* (University of Szeged, 2025) so that it runs end to end on free cloud hardware and can be inspected by others, for example as part of PhD applications. We follow the standard research method:

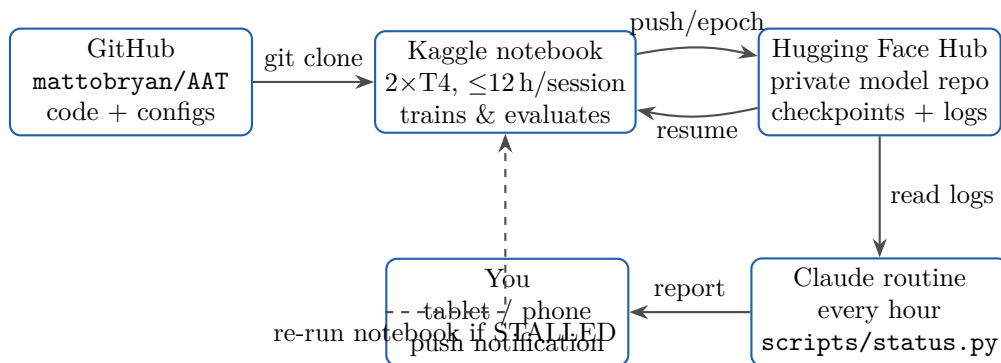
1. **Reproduce the baseline** (RAMP, Jiang & Singh, NeurIPS 2024) with the authors’ official code until our numbers match the published ones. This gives us RAMP’s weights.
2. **Lock the baseline**: the reproduced numbers and checkpoints are fixed and stored on the Hugging Face Hub.
3. **Build AAT on that baseline** and compare it with RAMP under the same starting model, training budget and evaluation.

Research question. Does feedback-driven scheduling of the perturbation budget ϵ (per class) and of the attacked ℓ_p norm improve *union robustness* over $\{\ell_1, \ell_2, \ell_\infty\}$, and robustness to threats not seen in training, compared with fixed-schedule multi-norm adversarial training (RAMP, E-AT, MAX)? All evaluation uses AutoAttack at fixed standard budgets.

	Hypothesis	Test
H1	Full AAT beats fixed-schedule multi-norm AT on union robustness	<code>aat_full</code> vs RAMP, E-AT, MAX; gain $> 2 \times$ seed std, clean drop ≤ 1 pp
H2	Class-feedback ϵ narrows the class robustness gap	worst-class union $\uparrow$, class std $\downarrow$
H3	Adaptive norm sampling beats cyclic sampling	<code>v2b_adaptive</code> vs <code>v2a_cyclic</code>
H4	AAT generalises better to unseen threats	larger- ϵ grid; ℓ_2 held out of training

2 System overview

Nothing needs to run on your laptop. Training runs on Kaggle’s servers, checkpoints and logs go to a private Hugging Face repository, and a scheduled Claude routine reads that repository and sends status reports to the Claude app on your phone or tablet.



3 Repository layout

4 Step 1: the RAMP baseline

4.1 Targets

Published values (%), AutoAttack, first 1,000 CIFAR-10 test points.

Path	Content
aat/models.py	PreActResNet-18 (ReLU), and the RAMP/E-AT softplus variant that loads the official <code>pretr_*.pth</code> weights (verified: identical logits, max diff 5×10^{-7})
aat/attacks.py	PGD under $\ell_\infty, \ell_2, \ell_1$ with per-sample budgets (training only)
aat/schedulers.py	adaptive ϵ (static, linear, norm/class feedback, per-sample) and norm schedulers
aat/losses.py	max / mean / hybrid multi-norm losses, RAMP KL logit pairing
aat/train.py	feedback $\rightarrow$ adapt $\rightarrow$ adversarial-training loop, resume + HF sync
aat/evaluate.py	AutoAttack (APGD-CE + APGD-T, official RAMP settings), union, per-class, unseen grid
aat/hub.py	Hugging Face push/pull, history squashing, never crashes training
baselines/ scripts/	published target numbers; resume/HF patch for the official <code>RAMP.py</code> <code>ramp_official.sh</code> , <code>compare_targets.py</code> , <code>status.py</code> , <code>aggregate.py</code> , <code>run_queue.sh</code>
configs/	one YAML per method (baselines and AAT ablations)
notebooks/	<code>ramp_baseline_kaggle.ipynb</code> (step 1), <code>aat_kaggle.ipynb</code> (step 2)
docs/	thesis audit, RAMP reproduction notes, this guide

Setting	Clean	ℓ_∞	ℓ_2	ℓ_1	Union	Role
RAMP from scratch, 80 ep, $\lambda=5$, GP (paper Tab. 3)	81.2	46.0	65.8	48.3	44.6	thesis baseline
Thesis Table 7.1 (5 runs)	81.3	45.96	65.74	48.42	44.5	to match
RAMP fine-tune 3 ep, $\lambda=1.5$ (paper Tab. 24)	81.1	45.4	66.1	47.2	43.1	cross-check
<code>pretr_Linf</code> (fine-tuning start)	83.7	48.1	59.8	7.7	≈ 7	sanity

4.2 What is run

The **official code** (uiuc-focal-lab/RAMP, pinned commit `be4971f`) is cloned during setup. Two changes are applied, both reviewable:

- `import copy` added to `utils.py` (the `-gp` path otherwise raises `NameError`);
- `baselines/ramp_resume_hub.patch` (≈ 90 lines): `-resume` restores both models, both optimisers and all RNG states; `-time_budget_h` exits cleanly after the last epoch that fits; `-hf_repo` pushes the resume checkpoint, logs and `ep_*.pth` after every epoch. The training algorithm is untouched.

Our evaluator then scores every checkpoint with AutoAttack **standard** settings restricted to APGD-CE and APGD-T (for ℓ_1 : 5 restarts, 5 target classes). Seed 0 is also scored by the official `eval.py` on the same points, as a check of our evaluator.

4.3 Acceptance

Per metric: **OK** if $|\Delta| \leq 1$ pp from the target, **CLOSE** if ≤ 2 pp (report with an explanation), otherwise **OFF** (investigate before building AAT).

5 One-time setup

5.1 Hugging Face

1. Create a free account at <https://huggingface.co>.
2. Settings $\rightarrow$ Access Tokens $\rightarrow$ *New token*, type **Write**. Copy it.
3. Choose a repository name, e.g. `<hf-username>/aat-checkpoints`. You do not need to create it: the first push creates it as **private**.

5.2 Kaggle

1. Verify your phone number (Settings), which is required for GPUs and internet access.
2. Create a notebook via *File* → *Import notebook*, choose GitHub, repository `mattobryan/AAT`, and select the file `notebooks/ramp_baseline_kaggle.ipynb`. You can also upload the `.ipynb` from a local clone.
3. *Add-ons* → *Secrets* → *Add secret*: label `HF_TOKEN`, value = the write token. Tick it for this notebook.
4. Session options: Accelerator **GPU T4 ×2**, Internet **on**, Persistence *Files only*.
5. In the first code cell, set `HF_REPO` to your repository name.

5.3 Monitoring (Claude routine)

The cloud environment of the Claude session that runs the routine needs two environment variables. Add them in the environment settings (session title bar → cloud environment → Edit), *never* in a chat message:

- `HF_TOKEN`: a Hugging Face token. A **read** token is enough here, so create a second, read-only token.
- `HF_REPO`: the same repository name as on Kaggle.

Until both are set, every report reads “monitor not configured”.

6 Running

6.1 A session

1. Open the notebook on Kaggle and click **Save Version** → **Save & Run All (Commit)**. The run continues on Kaggle’s servers after you close the browser, up to 12 hours.
2. Setup cell: clones the repository and the official RAMP code, applies the patch, installs dependencies and downloads CIFAR-10 (≈ 3 min).
3. Sanity cell: evaluates `pretr_Linf`. Expect $\approx 83.7 / 48.1 / 59.8 / 7.7 / \text{union} \approx 7$.
4. Stage A: RAMP from scratch, seed 0 on GPU 0 and seed 1 on GPU 1. Each run stops at 11 h and pushes its checkpoint.

6.2 Resuming

When a report says **STALLED**, or the Kaggle version shows *complete* while a run is not **DONE**, run the notebook again (same button). It pulls `resume.pth` from the Hub and continues from the last finished epoch. Finished runs are recognised and skipped. Repeat until both seeds are **DONE**, then change the seed lists to “2” “3” and later “4”.

6.3 Evaluating and comparing

After the seeds finish, the evaluation cell scores them and prints the comparison:

```
bash scripts/ramp_official.sh eval ramp_scratch "0 1" 0
python scripts/compare_targets.py --runs runs_official --table thesis_table7_1 --map
    ramp_scratch_official=ramp
python scripts/compare_targets.py --runs runs_official --table scratch_table3 --map
    ramp_scratch_official=ramp_l5
```

Stage B (the fine-tuning cross-check, ≈ 1 GPU-hour per seed) can run on leftover session time.

7 Monitoring and reports

The routine runs every hourutes in a fresh cloud session. It checks out the branch, runs `python scripts/status.py`, and sends a short report to the Claude app. Example:

```
AAT status 2026-09-26 14:30 UTC (user/aat-checkpoints)
- ramp_scratch_15_s0: running | epoch 37/80 | 15 min/epoch | last push 6 min ago | ETA
  10.8 GPU-h
  Linf 44.0% L2 63.5% L1 46.0% clean 78.5% union 42.5%
- ramp_scratch_15_s1: STALLED - re-run the notebook | epoch 41/80 | ...
```

State	Meaning and action
running	a push arrived within $2\times$ the mean epoch time + 20 min. Nothing to do.
STALLED	no push for longer. Usually the 11 h budget or the 12 h limit ended the session: re-run the notebook. If it happens within minutes of starting, open the Kaggle log (likely a setup error).
DONE	all epochs finished. Run the evaluation cell.
monitor not configured	HF_TOKEN / HF_REPO are missing from the routine's environment.

The in-training numbers in the report come from 200 points and 100-step APGD. They are for tracking only. Reported results come exclusively from the final AutoAttack evaluation.

8 Step 2: AAT experiments

Once the baseline is locked, AAT runs through `aat/train.py` with the same HF sync and time budget. The primary comparison starts AAT from the same model as RAMP, with the same budget and the same evaluation, so any difference is due to adaptive scheduling. Continuing from the RAMP weights is optional and needs a control in which RAMP also continues for the same number of epochs.

Config	Thesis name	What changes
v3a_max	3A	all three norms, fixed ϵ , per-sample worst case
v1a_linear	1A	+ global linear ϵ ramp
v1b_sample	1B	+ per-sample, class-aware ϵ (gradient norm + class robustness)
v2a_cyclic / v2b_adaptive	2A / 2B	one norm per batch: round-robin vs Boltzmann on feedback
v3b_classeps	3B	class-feedback ϵ controller, Eq. (4.19)
aat_full	full AAT	+ adaptive norm weights, hybrid loss, KL pairing
aat_full_no_l2	unseen norm	full AAT trained without ℓ_2

Before AAT results are trusted, `configs/ramp.yaml` (our reimplementaion) must reproduce the locked official RAMP numbers. That check comes first in step 2.

9 Evaluation protocol

- Fixed budgets only: $\ell_\infty = 8/255$, $\ell_2 = 0.5$, $\ell_1 = 12$. Adaptive ϵ is never used at test time.
- AutoAttack APGD-CE + APGD-T, first 1,000 test points (the protocol of the RAMP fine-tuning table, confirmed by the 83.7% clean accuracy of `pretr_Linf`).
- Union = per-sample AND over the three norms (not the minimum of the three accuracies).
- Unseen threats: $\ell_\infty \in \{4, 12, 16\}/255$, $\ell_2 \in \{0.25, 1, 1.5\}$, $\ell_1 \in \{6, 18, 24\}$, and ℓ_2 for runs trained without it.

- The test set is never used for adaptation or model selection. The last checkpoint is always used.

10 Budget and schedule

Kaggle gives about 30 GPU-hours per week and at most 12 hours per session. The figures below are estimates: the first logged epoch time replaces them.

Job	est. T4 time	sessions
Setup + <code>pretr_Linf</code> sanity	15 min	–
RAMP from scratch, 1 seed (80 ep, GP, fp32)	15–25 h	2–3
two seeds in parallel (2×T4)	same wall time	2–3
RAMP fine-tune cross-check, 5 seeds	≈5 GPU-h	1
AutoAttack eval, 1,000 points, per model	10–30 min	–

Note. Five from-scratch seeds cost more than one week’s quota. Plan: seeds 0–1 in week 1, together with the fine-tuning cross-check; seeds 2–4 in week 2, in parallel with the first AAT runs. Two seeds already give a mean and spread to compare with the thesis Table 7.1.

11 Troubleshooting

Symptom	Fix
<code>[hub] WARNING push failed</code>	transient network or rate limit; the next epoch pushes again. If it repeats, check that the token has <i>write</i> scope and that <code>HF_REPO</code> is spelled correctly.
nothing to pull on the first run	expected: no checkpoint exists yet.
Setup fails on <code>git apply</code>	the patch is already applied (the setup script skips it) or the pinned commit changed; re-run setup in a fresh session.
CUDA out of memory	another run is using the same GPU: check <code>CUDA_VISIBLE_DEVICES</code> in the cell (0 and 1).
Sanity numbers far from 83.7 clean / 48.1 ℓ_∞	evaluation or data problem: stop and investigate before training.
Result OFF by > 2 pp	compare the seed-0 official eval log with ours; check the epoch count and λ in <code>log_train.txt</code> .

12 Checklist

- ☐ HF write token created; Kaggle secret `HF_TOKEN` added and enabled for the notebook
- ☐ `HF_REPO` set in the notebook’s first cell
- ☐ Read-only HF token and `HF_REPO` added to the Claude environment (for reports)
- ☐ Session 1: Save & Run All; the sanity numbers match
- ☐ Re-run on every **STALLED** report until seeds 0–1 are **DONE**
- ☐ Evaluate, then run `compare_targets.py`: all OK or **CLOSE** $\Rightarrow$ baseline locked
- ☐ Step 2: `configs/ramp.yaml` reproduces the locked baseline, then the AAT runs start

A Command reference

```
# official baseline (inside the Kaggle notebook or any GPU shell)
export HF_REPO=<hf-username>/aat-checkpoints HF_TOKEN=... # token via Kaggle secret on
Kaggle
bash scripts/ramp_official.sh setup
bash scripts/ramp_official.sh pretr 0
bash scripts/ramp_official.sh train ramp_scratch "0" 0 # resumes automatically
bash scripts/ramp_official.sh eval ramp_scratch "0 1" 0
bash scripts/ramp_official.sh train ramp "0 1 2 3 4" 0 # fine-tuning cross-check

# AAT (step 2)
python -m aat.train --config configs/aat_full.yaml --set seed=0
python -m aat.evaluate --run runs/aat_full_s0
python scripts/aggregate.py --runs runs

# status from anywhere
python scripts/status.py
```